

```
!pip install -q librosa soundfile numpy matplotlib scipy
```

```
import librosa
import librosa.display
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import Audio, display
from scipy.signal import correlate
```

```
eps = 1e-10
```

```
import requests
import os
import tempfile
```

```
original_url = "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3" # Updated to a working URL
karaoke_url = "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-2.mp3" # Updated to a working URL
different_url = "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-3.mp3" # Updated to a working URL
```

```
sr = 22050 # sampling rate
```

```
def download_audio(url, temp_dir):
    filename = url.split('/')[-1]
    local_path = os.path.join(temp_dir, filename)
    response = requests.get(url)
    response.raise_for_status() # Raise an exception for HTTP errors
    with open(local_path, 'wb') as f:
        f.write(response.content)
    return local_path
```

```
# Create a temporary directory to store downloaded audio files
with tempfile.TemporaryDirectory() as tmpdir:
    print(f"Downloading files to temporary directory: {tmpdir}")
    original_local_path = download_audio(original_url, tmpdir)
    karaoke_local_path = download_audio(karaoke_url, tmpdir)
    different_local_path = download_audio(different_url, tmpdir)

    original, _ = librosa.load(original_local_path, sr=sr, mono=True)
    karaoke, _ = librosa.load(karaoke_local_path, sr=sr, mono=True)
    different, _ = librosa.load(different_local_path, sr=sr, mono=True)

    # Trim to same length
    minlen = min(len(original), len(karaoke), len(different))
    original = original[:minlen]
    karaoke = karaoke[:minlen]
    different = different[:minlen]

    print("All audio loaded, duration:", minlen/sr, "seconds")
```

```
Downloading files to temporary directory: /tmp/tmp61dxrg_6
All audio loaded, duration: 344.08489795918365 seconds
```

```
def normalized_corr(x, y):
    corr = np.sum(x * y)
    norm = np.sqrt(np.sum(x**2) * np.sum(y**2)) + eps
    return corr / norm
```

```
corr_ok = normalized_corr(original, karaoke)
corr_od = normalized_corr(original, different)
corr_kd = normalized_corr(karaoke, different)

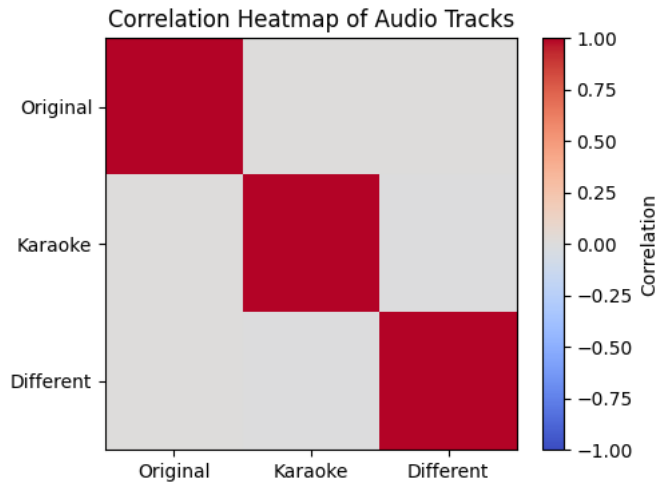
# Put into matrix
corr_matrix = np.array([
    [1,      corr_ok, corr_od],
    [corr_ok, 1,      corr_kd],
    [corr_od, corr_kd, 1      ]
])
```

```
print("\n--- Correlation Values (Normalized) ---")
print("Original vs Karaoke: ", corr_ok)
print("Original vs Different Song:", corr_od)
print("Karaoke vs Different Song:", corr_kd)
```

```
--- Correlation Values (Normalized) ---
```

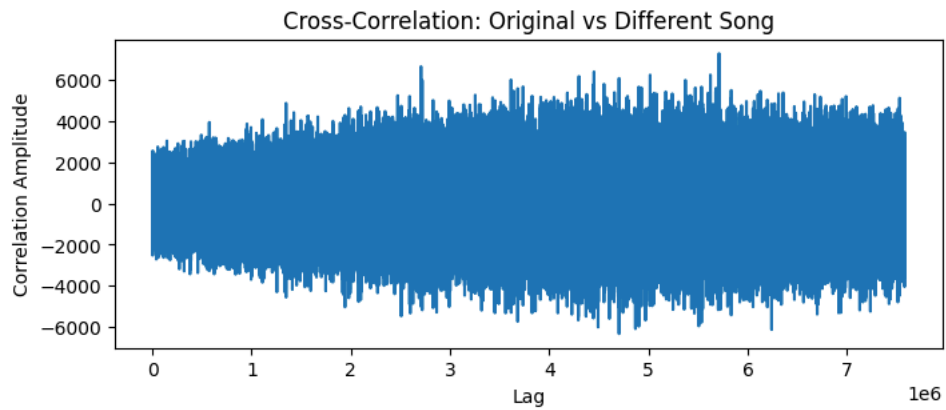
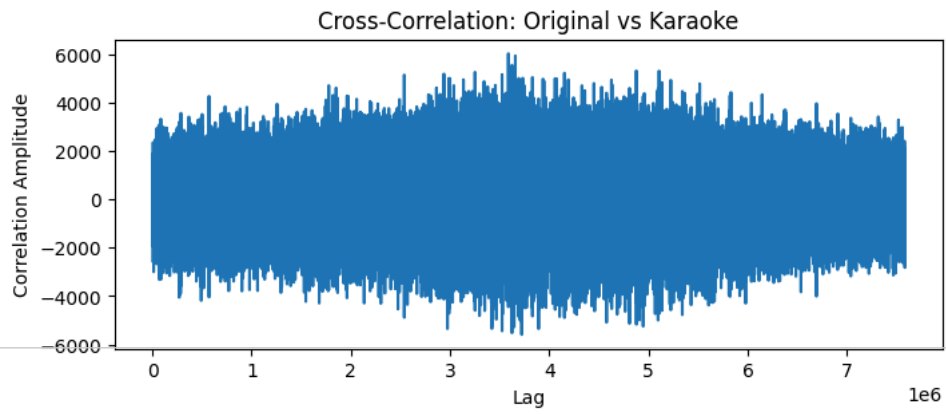
Original vs Karaoke: 0.0027838214
 Original vs Different Song: 0.0006761726
 Karaoke vs Different Song: -0.0014690085

```
plt.figure(figsize=(5,4))
plt.imshow(corr_matrix, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar(label="Correlation")
plt.xticks([0,1,2], ["Original", "Karaoke", "Different"])
plt.yticks([0,1,2], ["Original", "Karaoke", "Different"])
plt.title("Correlation Heatmap of Audio Tracks")
plt.show()
```



```
def plot_cross_corr(a, b, title):
    c = correlate(a, b, mode='same')
    plt.figure(figsize=(8,3))
    plt.plot(c)
    plt.title(title)
    plt.xlabel("Lag")
    plt.ylabel("Correlation Amplitude")
    plt.show()

plot_cross_corr(original, karaoke, "Cross-Correlation: Original vs Karaoke")
plot_cross_corr(original, different, "Cross-Correlation: Original vs Different Song")
plot_cross_corr(karaoke, different, "Cross-Correlation: Karaoke vs Different Song")
```



Cross-Correlation: Karaoke vs Different Song